

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA51 **COURSE NAME:** Cryptography and Network Security

COURSE OUTCOME COVERED

CO4: *Implement best security practices for IP, email, and web service using PGP, S/MIME for enhancing email Security.CO (BL3, BL6)*

Assessment Tool Weightage: 40%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A

Coding Challenge

Code of Conduct:

I Eddu Suchier/192524070 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: E. Suchier

Annexure A

Coding Challenge

1. Write a program to simulate the IPSec Authentication Header (AH) process that generates and verifies a message authentication code (MAC) for IP packet integrity.

Python Program:

```
import hashlib
import hmac

# Shared secret key
key = b"secret_key"

# Simulated IP packet fields
source_ip = "192.168.1.10"
destination_ip = "192.168.1.20"
protocol = "TCP"
data = "Hello, this is a secure IP packet."

# Create packet data
packet = source_ip + destination_ip + protocol + data

# Generate AH Authentication Data (HMAC)
def generate_mac(packet):
    return hmac.new(
        key,
        packet.encode(),
        hashlib.sha256
    ).hexdigest()

# Generate MAC
mac = generate_mac(packet)

print("Original IP Packet")
print("-----")
print("Source IP      :", source_ip)
print("Destination IP :", destination_ip)
print("Protocol       :", protocol)
```

```

print("Data      :", data)
print("AH MAC    :", mac)
# Verify original packet
received_mac = generate_mac(packet)
print("\nVerification of Original Packet:")
if hmac.compare_digest(mac, received_mac):
    print("Authentication SUCCESS")
    print("Packet integrity verified.")
else:
    print("Authentication FAILED")
# Simulate packet modification
modified_data = "Modified packet data!"
modified_packet = source_ip + destination_ip + protocol + modified_data
# Calculate MAC for modified packet
modified_mac = generate_mac(modified_packet)
print("\nModified Packet")
print("-----")
print("Modified Data :", modified_data)
print("New AH MAC   :", modified_mac)
# Verify modified packet against original MAC
print("\nVerification of Modified Packet:")
if hmac.compare_digest(mac, modified_mac):
    print("Authentication SUCCESS")
    print("Packet is valid.")
else:
    print("Authentication FAILED")
    print("Packet modification detected!")

```

Working

- The program creates a simulated IP packet containing source IP, destination IP, protocol, and data.
- HMAC-SHA256 generates the Authentication Data, simulating the AH integrity check.
- The receiver calculates the HMAC again and compares it with the received value.

- If an attacker modifies the packet, the HMAC changes.
- The comparison fails, indicating that the packet integrity has been compromised.

5. Create a program to classify emails as spam or legitimate using email header analysis and content-based filtering techniques.

Python Program:

Email Spam Detection using Header and Content Analysis

```
spam_keywords = [
    "win", "prize", "lottery", "free", "offer",
    "urgent", "click", "money", "claim", "congratulations"
]

def classify_email(sender, subject, content):
    score = 0
    # Header Analysis
    suspicious_domains = ["spam.com", "fakeoffer.com", "lottery.net"]
    domain = sender.split("@")[-1]
    if domain in suspicious_domains:
        score += 3
    # Subject Analysis
    subject_lower = subject.lower()
    for word in spam_keywords:
        if word in subject_lower:
            score += 2
    # Content Analysis
    content_lower = content.lower()
    for word in spam_keywords:
        if word in content_lower:
            score += 1
    # Classification
    if score >= 5:
        return "SPAM", score
    else:
```

```

        return "LEGITIMATE", score
# Sample Emails
emails = [
    {
        "sender": "promo@spam.com",
        "subject": "Congratulations! You Won a Free Prize",
        "content": "Click here to claim your money and free gift."
    },
    {
        "sender": "admin@college.edu",
        "subject": "Exam Schedule Released",
        "content": "The semester examination timetable has been published."
    },
    {
        "sender": "offers@fakeoffer.com",
        "subject": "Limited Time Offer",
        "content": "Urgent! Claim your lottery prize now."
    }
]

# Process Emails
for i, email in enumerate(emails, start=1):
    result, score = classify_email(
        email["sender"],
        email["subject"],
        email["content"]
    )
    print(f"\nEmail {i}")
    print("-" * 40)
    print("Sender :", email["sender"])
    print("Subject:", email["subject"])
    print("Result :", result)
    print("Score :", score)

```

Explanation

Header Analysis

- Checks the sender's domain.
- Suspicious domains increase the spam score.

Content-Based Filtering

- Searches for common spam keywords such as:
 - Free
 - Prize
 - Lottery
 - Money
 - Click
 - Offer
 - Urgent

Classification Rule

- Score $\geq 5 \rightarrow$ Spam
- Score $< 5 \rightarrow$ Legitimate

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases